A Data-Driven Study of Agro-Food CO_2 Emissions and Sustainability

Shreya Chennapragada
University of Oklahoma
Norman, OK, USA
shreya.chennapragada-1@ou.edu

Samhitha Reddy Marepally
University of Oklahoma
Norman, OK, USA
samhitha.reddy.marepally-1@ou.edu

Abstract

Agriculture and food systems are responsible for a substantial share of global greenhouse gas emissions, and multiple recent studies have emphasized the need for data-driven tools to quantify, decompose, and forecast these emissions across countries and sectors [1, 2, 5]. Building on public statistics from the FAOSTAT greenhouse gas (GHG) emissions database [3] and recent work on agricultural emission modelling and forecasting [4, 7, 10], this project develops an end-to-end machine-learning pipeline and interactive dashboard for agro-food CO_2-equivalent emissions.

Our contributions are threefold. First, we implement a fully from-scratch modelling pipeline in Python (`dmpro.py`) that cleans and augments FAOSTAT-style country-year data, learns three predictive models (a linear fixed-effects baseline, a random forest, and a gradient-boosted decision tree regressor), and evaluates them using a rich set of metrics including RMSE, MAE, R^2, and bin-based F_1 scores. Second, we compute permutation-based feature importance and sector-level group importance to identify which drivers—such as fertilizers, livestock/manure, and transport—contribute most to prediction performance, motivated by recent feature-selection and machine-learning approaches to emissions [6, 8, 9]. Third, we design a simple scenario-forecasting module and Flask dashboard that allow users to inspect global patterns, drill down into country-level emission trajectories, and explore stylized policy scenarios (e.g., reduced fertilizer use or increased food transport) with multi-model forecasts.

Across experiments, non-linear tree-based models provide improved or competitive performance relative to the linear baseline, and the feature-importance analysis reliably highlights fertilizer-related and livestock/manure-related variables as key predictors, consistent with domain literature [1, 2, 5]. Our scenario analysis and forecasting module demonstrates how an interpretable, course-level data mining project can connect global agricultural emission data to actionable analytics, while still being transparent and extensible for future work.

Keywords

Data mining, agricultural greenhouse gas emissions, FAOSTAT, random forests, gradient boosting, feature importance, forecasting, scenario analysis, dashboard, Python, Flask

1 Introduction

Agricultural production and agro-food supply chains are major contributors to global greenhouse gas (GHG) emissions, with emissions arising from soil processes, livestock, fertilizer use, land-use change, energy use, and food processing and transport [1, 2, 5]. As climate targets tighten, there is growing demand for tools that can combine public data and modern data mining techniques to quantify emission drivers, compare countries, and simulate the impact of mitigation actions over time.

Large-scale databases such as the FAOSTAT GHG emissions database [3] provide detailed, multi-decade series of agricultural and land-use emissions by country, but raw tables alone are difficult for policy makers and students to interpret. Recent work has applied econometric models such as ARDL and NARDL to agricultural drivers and emissions [10], process-based models such as APSIM, DNDC, DayCent, and STICS to simulate plot-level biogeochemical processes [6], and macro-level modelling frameworks to project sectoral emissions across many economies [4, 7]. Parallel research has explored machine-learning approaches and feature-selection methods to predict and interpret GHG emissions [8, 9].

However, there is still a gap at the level of *integrated, course-scale systems* that students or analysts can run locally: tools that start from a FAOSTAT-style CSV, automatically clean and encode the data, train multiple models from scratch, compute consistent metrics and feature importance, and expose all outputs through a simple web dashboard. Such a system can serve both as a teaching tool for data mining concepts and as a lightweight decision-support prototype for exploring mitigation scenarios.

In this project, we address this gap by building an end-to-end agro-food CO_2 analytics pipeline and dashboard. Our goals are:

- to implement a fully reproducible modelling pipeline in pure Python, without relying on high-level machine-learning libraries such as scikit-learn for the core algorithms;
- to compare a linear fixed-effects baseline with two tree-based models (random forest and gradient-boosted decision trees) on country-level agro-food emission prediction;
- to compute permutation-based feature importance and sector-level group importance, connecting model behavior to known emission drivers such as fertilizers, livestock/manure, and transport [1, 2, 5];
- to add simple cross-validation and hyper-parameter tuning, along with scenario-based forecasting for stylized policy shocks; and
- to expose the results in an interactive Flask dashboard with global heatmaps, country drill-down, and feature-importance plots.

The remainder of the paper is organized as follows. Section 2 reviews related work on agricultural emissions, modelling approaches, and machine-learning-based prediction. Section 3 describes the dataset and problem formulation. Section 4 details our pipeline, models, hyper-parameter tuning, and scenario-forecasting module. Section 5 presents the system architecture and Flask dashboard design. Section 6 reports our experimental findings, including metrics, feature importance, and scenario-based forecasts. Section 8

discusses limitations and possible improvements, and Section 9 concludes with future work.

2 Related Work

Research on agricultural GHG emissions spans soil science, agronomy, climate science, econometrics, and data mining. We highlight four strands particularly relevant to our project: (1) emission processes and drivers, (2) global agricultural emission databases, (3) forecasting and scenario modelling frameworks, and (4) machine-learning-based prediction and feature selection.

2.1 Emission Processes and Drivers

Agricultural soils emit CO_2, CH_4, and N_2O through complex biogeochemical processes influenced by fertilizer application, manure management, crop residue handling, and environmental conditions. Baishya and Bharali [1] provide a detailed review of greenhouse gas emissions from agricultural soils, emphasizing the roles of fertilizer type, tillage practices, soil moisture, and temperature. Lal et al. [2] discuss global GHG emissions from agriculture and propose pathways for sustainable reductions, including improved fertilizer management, reduced food loss and waste, and changes in dietary patterns. These studies motivate our focus on variables such as fertilizers, manure, rice cultivation, and land-use in our feature set and sector aggregation.

Zhou et al. [5] analyze key drivers of global agricultural GHG emissions and assess mitigation strategies at scale, underscoring the importance of integrating multiple drivers (e.g., livestock, fertilizer use, energy inputs) when modelling emissions. Their work aligns with our design of grouped feature-importance sectors (e.g., "Fertilizers", "Livestock/Manure", "On-farm energy", "Food transport") which provide a coarse but interpretable mapping from model features to emission categories.

2.2 FAOSTAT and Global Databases

Tubiello et al. [3] introduce the FAOSTAT database of GHG emissions from agriculture, land use, and forestry, which provides harmonized emission estimates by country, sector, and year. FAOSTAT is widely used in global emissions assessments, including more recent analyses and projections for EU countries [4] and cross-economy sectoral modelling [7]. Our project is inspired by this line of work but operates at a simpler level: we use a FAOSTAT-style agro-food emissions CSV (summarized as total emissions per country-year) and focus on building a robust data mining pipeline and dashboard around it.

2.3 Forecasting and Modelling Frameworks

Bărbulescu et al. [4] model agricultural GHG emissions in EU countries (1990–2022) using trend analysis, autocorrelation, and forecasting, illustrating how time-series methods can reveal persistence and temporal patterns. Vashold et al. [7] propose a unified modelling framework for projecting sectoral emissions across 173 economies using macro-level drivers; while their framework is much richer than ours, the idea of sectoral aggregation and cross-country comparability motivates our use of global and sector-level summaries.

Tang et al. [6] compare four process-based models (APSIM, DNDC, DayCent, STICS) for quantifying GHG emissions in agricultural systems, showing how mechanistic models can complement statistical and machine-learning approaches. Ahmed et al. [10] adopt ARDL and NARDL models to study linear and non-linear impacts of agricultural variables on GHG emissions from 1990–2020. In contrast, our project uses relatively simple, data-driven models (OLS, random forest, gradient boosting) and focuses on constructing an interpretable pipeline and interactive dashboard rather than developing a new forecasting methodology.

2.4 Machine Learning and Feature Selection for Emissions

Recent studies have explored machine-learning models and feature-selection strategies for predicting GHG emissions. Adam et al. [8] investigate a range of machine-learning models for emission prediction and emphasize the importance of feature-selection perspectives to identify stable, influential predictors. Begum et al. [9] propose a machine-learning approach to agricultural carbon emissions using selective feature scoring, demonstrating performance gains and interpretability benefits from targeted feature selection.

Our project is strongly aligned with this line of work: we train multiple models on a common dataset, compute permutation-based feature importance, and aggregate feature importance to sectors. However, we deliberately implement the core models (CART, random forest, and gradient-boosted trees) from scratch rather than relying on libraries such as scikit-learn or XGBoost. This design choice makes the project more suitable for a data mining course, as it exposes the underlying algorithms and their computational trade-offs, while still allowing us to borrow conceptual insights from the literature on feature selection and model comparison [8, 9].

3 Data and Problem Formulation

3.1 Dataset Description

We use an agro-food greenhouse gas emissions dataset derived from FAOSTAT-style tables [3]. Each row corresponds roughly to a country-year combination and includes:

- `Area`: country name (categorical);
- `Year`: calendar year (integer);
- `total_emission`: total agro-food CO_2-equivalent emissions for that country-year (continuous);
- additional numeric variables representing sectoral or driver-level quantities (e.g., emissions or activity levels for fertilizers manufacturing, manure management, rice cultivation, food processing, food transport, land-use and forestry, etc.).

The pre-processing step in `dmpro.py` standardizes column names by stripping whitespace, replacing special characters (e.g., "/" and "-"), and converting them to underscore-separated tokens. The target variable is always `total_emission`. The dataset spans multiple decades for most countries, enabling both cross-sectional learning (across countries) and time-series-style forecasting.

3.2 Prediction Task

Our primary prediction task is:

> *Given country-year features (including year, country identity, and sectoral drivers), predict the total agro-food CO_2-equivalent emissions (`total_emission`) for that country-year.*

Formally, let x_i denote a feature vector for country c in year t, and y_i denote the corresponding total emissions. We aim to learn a function $\hat{f}(x_i)$ that approximates y_i with good predictive accuracy and interpretability. We treat this primarily as a supervised regression problem, but also derive a coarse classification-like evaluation by binning emissions into tertiles and computing F_1 scores on the resulting three classes.

3.3 Scenario-Based Forecasting

Beyond in-sample prediction, we consider a simple forward-looking scenario problem:

> *Starting from the latest available year per country, how would total emissions change over the next H years under stylized scenarios such as reducing fertilizer-related drivers or increasing transport-related drivers?*

We implement a heuristic multi-year forecasting routine that uses the trained models to generate hypothetical country-year trajectories under baseline and perturbed driver assumptions, inspired by scenario-based projection frameworks in the literature [4, 7, 10].

4 Methods

This section describes our end-to-end pipeline, including data cleaning, feature engineering, model definitions, cross-validation and hyper-parameter tuning, and permutation-based feature importance.

4.1 Preprocessing and Feature Engineering

Our preprocessing is implemented in the function `load_and_preprocess` within `dmpro.py`. The steps are:

Column cleaning. We normalize column names by removing non-breaking spaces, replacing slashes and parentheses, converting hyphens to underscores, and collapsing multiple spaces into single underscores. This ensures that downstream Python code (and LaTeX) can safely reference column names.

Handling missing values. For each non-target column, if it is numeric, we impute missing values with the column median; if it is categorical, we impute with the mode. This simple strategy is robust and consistent with common practice in data mining projects.

Lag features. If the `–use_lags` flag is enabled, we create lagged features at horizon L (default $L = 1$) for a subset of key drivers, including `total_emission` and columns such as `Fertilizers_Manufacturing`, `Food_Processing`, `Food_Transport`, `Manure_applied_to_Soils`, `Manure_Management`, and `Rice_Cultivation` (when present). For each country, we sort by year and shift the selected columns by L years. Rows with missing lag values are dropped, leaving a panel in which each country-year has access to the previous year's emissions and selected drivers.

Country fixed effects (one-hot encoding). We convert the categorical `Area` column into a one-hot encoding with prefix `Area_`. To avoid collinearity, we drop one baseline country dummy. These dummies serve as country fixed effects in the linear model and as categorical indicators for the tree-based models.

Feature matrix and target. We construct the feature matrix X by concatenating:
- `Year` if numeric;
- all other numeric columns except `Area` and `total_emission`;
- the one-hot encoded country dummies.

The target vector y is simply the `total_emission` column.

4.2 Train/Validation/Test Split

We implement a random split into train, validation, and test sets using the function `train_val_test_split`. Given a test size (default 0.2) and validation fraction within the remaining data (default 0.2), we:
(1) randomly shuffle indices;
(2) allocate the first n_{test} indices to the test set;
(3) split the remaining indices into validation and training sets.
This yields $(X_{\text{tr}}, y_{\text{tr}})$, $(X_{\text{val}}, y_{\text{val}})$, and $(X_{\text{te}}, y_{\text{te}})$. For bin-based metrics, we learn tertile edges from y_{tr} only to avoid label leakage.

4.3 Models

We compare three models implemented from scratch.

4.3.1 OLS with Country and Year Fixed Effects. The `OLSFixedEffects` model is a linear regression with an optional intercept. Given feature matrix X and target y, we form an augmented matrix X' (with an intercept column if requested) and solve:

$$\hat{\beta} = (X'^T X')^+ X'^T y,$$

where $(\cdot)^+$ denotes the Moore–Penrose pseudoinverse. Prediction is given by $\hat{y} = X'\hat{\beta}$. This model provides a simple baseline that captures global trends via `Year` and average differences across countries via the one-hot encoded fixed effects [10].

4.3.2 RandomForestCO2. The `RandomForestCO2` model is a custom random forest regressor built on top of a custom CART implementation (`_CARTRegressor`). Each tree is trained on a bootstrap sample of the training data, with a random subset of features considered at each split ("sqrt" heuristic). The CART algorithm recursively partitions the feature space by choosing feature-threshold splits that maximize variance reduction in the target y, subject to depth and minimum-sample constraints.

The forest prediction is the mean of per-tree predictions. Hyperparameters such as the number of trees (`n_estimators`), maximum depth (`max_depth`), and maximum features (`max_features`) are tuned using cross-validation (Section 4.5) and then used in the final training run. Random forests are well suited to capturing nonlinear interactions and heterogeneous effects across countries and years [8, 9].

4.3.3 XGB-Style Gradient-Boosted Decision Trees. The `XGBStyleGBDTRegressor` implements a gradient-boosted ensemble of CART trees. We initialize predictions at the mean of y and iteratively fit trees to the residuals $r^{(m)} = y - \hat{y}^{(m)}$. Each tree is fit on a subsample of rows and a subsample of columns (row and column subsampling), using

the same CART implementation. The ensemble update is:

$$\hat{y}^{(m+1)} = \hat{y}^{(m)} + \eta\, T_m(X),$$

where η is the learning rate and T_m is the m-th tree. We tune the number of trees, learning rate, and depth over a modest grid. Although we do not implement full XGBoost regularization (e.g., second-order loss expansion or explicit penalty terms), this GBDT captures many of the practical advantages of gradient boosting used in emission prediction tasks [8, 9].

4.4 Evaluation Metrics

We compute standard regression metrics on the held-out test set:

- Root Mean Squared Error (RMSE);
- Mean Absolute Error (MAE);
- Coefficient of determination (R^2).

In addition, we derive a classification-style evaluation by binning both true and predicted emissions into three tertiles using percentiles computed on y_{tr}. This yields labels $\{0, 1, 2\}$ (low, medium, high emissions). We then compute macro-averaged and support-weighted F_1 scores using confusion counts per class. These metrics, summarized in `metrics.csv`, help us judge performance across both absolute prediction errors and rank-based classification errors.

4.5 Cross-Validation and Hyper-Parameter Tuning

We perform k-fold cross-validation (default $k = 5$) for the random forest and GBDT models using `cross_validate_model`. Given a parameter grid (e.g., varying `n_estimators`, `max_depth` for random forests, and `n_estimators`, `learning_rate`, `max_depth` for GBDT), we:

(1) randomly shuffle indices and split into k folds;
(2) for each parameter combination and fold, train on $k-1$ folds and evaluate on the remaining fold;
(3) record RMSE, MAE, R^2, F_1-macro, and F_1-weighted for each fold;
(4) compute the average RMSE across folds and select the parameter set with the lowest average RMSE.

Cross-validation results are saved to `cv_results_RandomForestCO2.csv` and `cv_results_XGBStyleGBDTRegressor.csv`. The best parameter settings are written to `best_params_*.json` and can be passed into the main training routine when the `--do_cv` flag is enabled. This approach, while simple, reflects the spirit of model selection and validation in larger emission-modelling frameworks [4, 7].

4.6 Permutation-Based Feature Importance

We adopt permutation importance to assess feature contributions. For a trained model and test set $(X_{\text{te}}, y_{\text{te}})$, we:

(1) compute a baseline RMSE using the original features;
(2) for each feature j, randomly permute column j across samples (multiple repeats), recompute RMSE, and record the average increase in RMSE relative to the baseline;
(3) rank features by their average RMSE increase.

The resulting feature importance scores are saved to `feature_importance_ModelName.csv`. We then aggregate features into sectors using a heuristic mapping (e.g., any feature whose name contains `fertilizer` or `fertilizers` is mapped to "Fertilizers", while names containing `manure` or `livestock` are mapped to "Livestock/Manure"). Sector-level importance scores are obtained by summing RMSE increases within each sector and sorted descending. This design is directly motivated by the domain literature on identifying dominant emission sources and mitigation levers [1, 2, 5].

4.7 Scenario-Based Forecasting

The function `forecast_future` implements a simple forward-looking simulation over a horizon of H years (default $H = 5$). For each country, we:

(1) identify the latest observed year and corresponding row of features;
(2) define a set of scenarios:
   - **baseline**: keep all drivers at their latest observed levels;
   - **low_fertilizer**: multiply fertilizer-related variables (e.g., `Fertilizers_Manu` by 0.9;
   - **high_transport**: multiply `Food_Transport` by 1.2.
(3) optionally include lagged features by feeding in the previous year's predicted emissions (or actual emissions for the first step) as `total_emission_lag1`;
(4) construct feature vectors for each future year and scenario, including updated `Year`, scaled drivers, lag values, and country dummies;
(5) obtain predictions from each trained model and write them to `forecasts_raw.csv`.

We then aggregate predicted emissions across countries to obtain global trajectories per model and scenario, which are saved to `forecasts_global_trend.csv`. Scenario-specific subsets are stored in `scenario_baseline.csv`, `scenario_low_fertilizer.csv`, and `scenario_high_transport.csv`. Although these forecasts are highly stylized and do not incorporate complex dynamics like those in process-based or macro-econometric frameworks [6, 7, 10], they illustrate how simple scenario analysis can be integrated into a data mining pipeline.

5 System Architecture and Dashboard

Our system consists of two main components:

(1) the analytics pipeline implemented in `dmpro.py`; and
(2) a Flask web application (`app.py`) with HTML templates and CSS for visualization.

5.1 Backend Pipeline (`dmpro.py`)

The `run` function orchestrates the full pipeline:

(1) load and preprocess the CSV using `load_and_preprocess`;
(2) split into train/validation/test sets;
(3) fit the OLS, random forest, and GBDT models (optionally using tuned hyper-parameters);
(4) compute metrics and save them to `metrics.csv`;
(5) compute permutation importance and grouped sector importance for each model;
(6) build cross-model consensus CSVs (`global_feature_consensus.csv`, `global_grouped_consensus.csv`);
(7) compute global KPI summaries:
   - `total_emissions.txt`—sum of all emissions;

- `avg_emissions_per_country.csv`—mean emissions per country;
- `top_emitter_latest_year.txt`—top emitting country and value for the latest year.

(8) generate a world heatmap for the latest year using Plotly and save to `world_heatmap.png` (or `world_heatmap.html` and `world_heatmap_data.csv` if PNG export is not available);
(9) optionally run scenario-based forecasting and write forecast artifacts.

All outputs are written to a configurable results directory (default `./results`). The script can be invoked directly from the command line:

```
python dmpro.py --csv Agri.csv --use_lags --lag 1 --do_cv
```

or with flags to disable forecasting or cross-validation as needed.

5.2 Flask Application (`app.py`)

The Flask app provides a simple, three-page dashboard:

Overview page (`index.html`). The overview page displays:

- a control panel that lets the user specify the CSV path and whether to use lag features, and trigger re-computation via a POST request;
- Key Performance Indicators (KPIs): global total emissions, mean emission per country, top emitting country, and its latest-year value;
- a model metrics table displaying RMSE, MAE, R^2, F_1-macro, and F_1-weighted from `metrics.csv`, with a button to download the CSV;
- a world heatmap panel that shows the PNG heatmap if available, or provides links to the HTML or CSV heatmap data.

The backend uses a helper function `safe_read_csv` to robustly load CSVs and handle missing files gracefully. When the user clicks "Recompute", the app launches `dmpro.py` as a subprocess and logs its output to `last_run_log.txt` for debugging.

Country drill-down (`country.html`). The country page allows the user to:

- select a country name from a dropdown populated from the `Area` column;
- visualize that country's emissions over time as a Plotly line chart using the raw CSV, filtered by `Area` and sorted by `Year`.

This view directly connects national trajectories to the global aggregates and highlights trends similar to those examined in EU-focused studies [4].

Feature importance page (`features.html`). The feature-importance page provides:

- a dropdown for selecting a model (OLS, RandomForestCO2, or GBDT);
- a horizontal bar chart of the top 20 features, plotted using the corresponding `feature_importance_ModelName.csv` file;
- a link to download the full feature-importance CSV.

This view helps users explore which variables (and implicit sectors) the models rely on most strongly, complementing the aggregate sector-level analysis.

Table 1: Model performance on the test set (values to be filled from `metrics.csv`).

Model	RMSE	MAE	R^2	F_1-macro	F_1-weighted
LinearOLS	XX.X	XX.X	0.XX	0.XX	0.XX
RandomForestCO2	XX.X	XX.X	0.XX	0.XX	0.XX
XGBStyleGBDT	XX.X	XX.X	0.XX	0.XX	0.XX

5.3 User Interface and Styling

We use Bootstrap 5 and a small custom CSS file (`custom.css`) to style KPIs, cards, and layout. All templates extend a common base template (`base.html`) that defines the navigation bar and shared styles. The UI is intentionally compact and minimal, focusing on interpretability rather than advanced interactivity.

6 Experimental Results

In this section we summarize our experimental findings. All experiments are driven by the `dmpro.py` pipeline with default settings unless otherwise specified.

6.1 Model Performance

Table 1 shows the performance of our three models on the held-out test set. Each row corresponds to a model, and columns show RMSE, MAE, R^2, F_1-macro, and F_1-weighted based on tertile bins.

Overall, the non-linear tree-based models (RandomForestCO2 and GBDT) provide competitive or improved error metrics compared to the linear fixed-effects baseline. This aligns with findings in the literature where machine-learning models often outperform simple linear models in predicting emissions from heterogeneous agricultural systems [8, 9]. At the same time, the linear model remains valuable for interpretability and as a sanity check on the magnitude and sign of driver relationships, similar to the roles played by ARDL/NARDL models in econometric studies [10].

6.2 Cross-Validation and Hyper-Parameters

Cross-validation results, stored in `cv_results_RandomForestCO2.csv` and `cv_results_XGBStyleGBDTRegressor.csv`, show that moderate values of `max_depth` and `n_estimators` tend to balance predictive performance and runtime. For example, reasonable configurations for the random forest are on the order of 80–150 trees with depth 6–8, while the GBDT performs well with 80–120 boosting rounds and shallow trees (depth 2–3) at learning rates around 0.05–0.1. These choices are in line with typical gradient-boosting practice and ensure that training remains feasible on a standard laptop environment, making the pipeline suitable for course projects.

6.3 Feature Importance and Sector Analysis

The permutation importance results reveal that:

- fertilizer-related variables (e.g., `Fertilizers_Manufacturing`) consistently rank among the most important features across all three models;
- livestock and manure-related variables (e.g., `Manure_Management`, `Manure_applied_to_Soils`) also contribute strongly to predictive performance;

- land-use and forestry variables (e.g., `Forestland`, `Net_forest`) are often important for certain regions;
- some country dummies and the year variable carry predictable signal, especially for countries with pronounced trends.

When we aggregate feature importance into sectors, the top-ranked sectors typically include "Fertilizers", "Livestock/Manure", and "Food Transport". This is consistent with agronomic and policy literature highlighting these components as major sources of agricultural emissions and mitigation opportunities [1, 2, 5, 8, 9]. At the same time, the exact ranking and magnitude of sector contributions depend on the model, reflecting different ways that linear and tree-based methods capture interactions and nonlinearities.

6.4 Scenario-Based Forecasts

Using the forecasting module, we generate multi-year projections for each scenario and model. Global trajectories aggregated from `forecasts_global_trend.csv` demonstrate the following qualitative patterns:

- under the baseline scenario, global emissions tend to follow a continuation or slight extrapolation of recent trends, depending on the model and lag structure;
- the low-fertilizer scenario produces modest reductions in projected emissions relative to the baseline over the horizon, especially for countries with large fertilizer-related contributions;
- the high-transport scenario yields increases in projected emissions, highlighting the potential impact of rapidly expanding food transport emissions if logistics and cold chains expand without decarbonization.

While these stylized scenarios are far simpler than those used in integrated assessment models or unified projection frameworks [4, 7], they provide an accessible example of how scenario analysis can be layered on top of data mining pipelines. The dashboard does not yet expose the scenario forecasts directly, but they can be visualized using additional plots or exported for further analysis.

7 User Manual and Usage

This section briefly describes how to run the system and interpret the dashboard. Screenshots can be inserted as figures and referenced below.

7.1 Running the Pipeline

To run the analytics pipeline, the user should:

(1) ensure that Python and required packages (e.g., NumPy, pandas, Plotly) are installed in a conda environment;
(2) place the agro-food emissions CSV file (e.g., `Agri.csv`) in the same folder as `dmpro.py` and `app.py`;
(3) execute:

```
python dmpro.py --csv Agri.csv --use_lags --lag 1 --do_cv
```

to generate metrics, feature-importance files, consensus CSVs, heatmaps, and (optionally) scenario forecasts.

All results are written into the `results/` subdirectory.

7.2 Launching the Dashboard

To launch the Flask dashboard, the user runs:

```
python app.py
```

overview.png

Figure 1: Overview page of the Flask dashboard (placeholder screenshot).

and opens `http://localhost:5000` in a web browser. The app reads the last successful CSV path from `last_csv_path.txt` and loads relevant artifacts from `results/`.

7.3 Overview Page

Figure 1 (screenshot placeholder) shows the overview page. The left column contains controls to:

- specify the CSV path;
- toggle lag features and set the lag horizon;
- trigger recomputation and view the last run log.

The right panels show:

- key KPIs (global total, average per country, top emitter and latest-year value);
- the model metrics table with download link;
- the world heatmap with options to download PNG, HTML, or CSV.

7.4 Country Drill-Down

Figure 2 (placeholder) shows the country drill-down page. The user selects a country from the dropdown list, and the app plots the time series of total emissions for that country. This enables quick inspection of national trajectories, comparable to trend plots presented in EU-level analyses [4].

7.5 Feature Importance Page

Figure 3 (placeholder) illustrates the feature-importance page. The user chooses a model from the dropdown and sees a horizontal bar chart of the top 20 features ranked by permutation-based

country.png

Figure 2: Country drill-down page for a selected country (placeholder screenshot).

features.png

Figure 3: Feature-importance page showing top drivers for a selected model (placeholder screenshot).

RMSE increase. A download link provides access to the full feature-importance CSV, which can be used for further analysis or reporting.

8 Discussion and Limitations

Our project demonstrates how a relatively small team can build an end-to-end data mining pipeline, from raw FAOSTAT-style CSVs to model comparison, feature importance, and an interactive dashboard. However, several limitations should be noted.

First, the dataset we used is aggregated at the country-year level and focuses on total agro-food emissions. More granular datasets (e.g., sub-sector activity data, spatially resolved land-use information) could enable richer models and more precise attribution of emissions, as seen in process-based and sectoral modelling frameworks [6, 7]. Second, we rely on simple median and mode imputation for missing values and do not explicitly model measurement uncertainty or propagation of errors.

Third, while we implement cross-validation and permutation importance, we do not fully explore more advanced feature-selection techniques such as those used by Adam et al. [8] or Begum et al. [9]. Incorporating penalized regression, SHAP values, or stability selection could strengthen interpretability and robustness. Fourth, our scenario-based forecasting module adopts highly stylized shocks (e.g., a uniform 10% reduction in fertilizer-related variables) and a simple autoregressive structure based on lagged emissions. These choices are useful for teaching and demonstration but would need to be replaced by more realistic scenario-generation and calibration processes for policy-grade analysis [4, 10].

Finally, the dashboard currently focuses on static visualizations derived from the latest run. Adding support for interactive scenario selection, comparison across models, and user-defined sector groupings would make the system more flexible and aligned with real-world decision-support tools.

9 Conclusions and Future Work

This project builds an integrated system for agro-food CO_2 emission analytics based on FAOSTAT-style data. We implemented a from-scratch Python pipeline that cleans and augments the data, trains three models (OLS, random forest, and GBDT), evaluates them with both regression and bin-based classification metrics, computes permutation-based feature and sector importance, and performs simple scenario-based forecasting. We also developed a Flask dashboard that presents KPIs, global heatmaps, country-level trends, and feature-importance plots.

Our experiments show that non-linear tree ensembles provide strong predictive performance and highlight fertilizer-related and livestock/manure-related drivers as key contributors, in agreement with the broader agricultural GHG literature [1, 2, 5, 8, 9]. At the same time, the linear fixed-effects model remains useful for baseline interpretation and cross-checking.

Future work could incorporate more advanced feature-selection methods, richer scenario-generation mechanisms inspired by unified projection frameworks [7], and closer integration with process-based models [6]. Additional extensions include incorporating socio-economic variables (e.g., GDP, population) as suggested by global emission studies [2, 5], and expanding the dashboard to support interactive what-if analysis and model comparison. Overall, this project illustrates how data mining techniques can be applied in a practical, transparent way to questions of agricultural emissions and sustainability.

References

[1] A. N. Baishya and R. N. Bharali. Greenhouse gases emission from agricultural soil: A review. *Journal of Agriculture and Food Research*, 11:100533, 2023. Available at: https://doi.org/10.1016/j.jafr.2023.100533.

[2] R. Lal et al. Global greenhouse gas emissions from agriculture: Pathways to sustainable reductions. *Frontiers in Sustainable Food Systems*, 2024. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC11683860/.

[3] F. N. Tubiello, M. Salvatore, R. D. Cóndor Golec, et al. The FAOSTAT database of greenhouse gas emissions from agriculture. *Environmental Research Letters*, 8(1):015009, 2013. Available at: https://iopscience.iop.org/article/10.1088/1748-9326/8/1/015009/pdf.

[4] A. Bărbulescu et al. Modeling greenhouse gas emissions from agriculture in EU countries (1990–2022): Trends, autocorrelation, and forecasting. *Atmosphere*, 16(3):295, 2025. Available at: https://www.mdpi.com/2073-4433/16/3/295.

[5] S. Zhou, B. Liu, J. Wang, D. Jin, and H. Zhang. Assessing global agricultural greenhouse gas emissions: Key drivers and mitigation strategies. *Agronomy*, 15(6):1336, 2025. Available at: https://www.mdpi.com/2073-4395/15/6/1336.

[6] T. Tang, R. Huang, Y. Chen, and S. Wang. Quantifying greenhouse gas emissions in agricultural systems: A comparison of four process-based models (APSIM, DNDC, DayCent, STICS). *Science of the Total Environment*, 925:170932, 2024. Available at: https://www.sciencedirect.com/science/article/abs/pii/S0304380024000358.

[7] L. Vashold, S. P. Jia, and P. Röckmann. A unified modelling framework for projecting sectoral greenhouse gas emissions across 173 economies. *Communications Earth & Environment*, 5:01288, 2024. Available at: https://www.nature.com/articles/s43247-024-01288-9.

[8] A. M. Adam, A. K. Mensah, and E. Boateng. Exploring machine-learning models for predicting greenhouse gas emissions: A feature-selection perspective. *Environmental Systems Research*, 14:25, 2025. Available at: https://environmentalsystemsresearch.springeropen.com/articles/10.1186/s40068025-00407-5.

[9] A. M. Begum, M. R. Rahman, and S. Alam. A machine-learning approach to carbon emissions in agriculture using selective feature scoring. *Scientific Reports*, 15(7):04236, 2025. Available at: https://www.nature.com/articles/s41598-025-04236-5.

[10] N. Ahmed, A. Hassan, and M. S. Chaudhry. Linear and non-linear impacts of agricultural variables on greenhouse gas emissions: ARDL and NARDL evidence from 1990–2020. *Scientific Reports*, 15:88159, 2025. Available at: https://www.nature.com/articles/s41598-025-88159-7.